

Web Design Principles

Basic Principles Involved in Developing a Website

1. Purpose and Planning

- Clearly define the goal of the website (Informational, e-commerce, portfolio, educational, etc.)
- Understand the target audience and their needs
- Plan the site structure, navigation flow and features before starting development
- Conduct thorough research before starting development

2. User Experience (UX)

- The website should be easy to navigate with logical menu and clear structure
- Ensure readability (appropriate fonts, spacing and colors)
- Provide fast loading speed and optimized performance
- Keep accessibility in mind (usable for all, including differently abled users)

3. User Interface (UI)

- Follow a consistent layout and theme across all pages
- Use responsive design so the site works on desktop, tablet and mobile
- Apply visual hierarchy

4. Content

- Content should be clear, relevant and updated regularly
- Use SEO-friendly content (keywords, metadata, headings, alt text for images)
- Search engine optimisation → SEO

5. Technology & Development

- Use appropriate technologies
 - HTML → structure of the site
 - CSS → styling and layout
 - JavaScript → interactivity and dynamic features
 - Backend languages/databases (PHP, Python, Node.js, MySQL etc.) for dynamic site

6. Security

- Protect the site with HTTPS (SSL certificate)
- Validate user inputs to prevent hacking
- Regularly update software, plugins and frameworks

7. Testing and Debugging

- Test for cross-browser compatibility (Chrome, Firefox, Safari, Edge)
- Test on different devices and screen sizes
- Fix broken links, errors and bugs.

8. Planning Process

1. Defining Purpose & Goals

- Why is the website being built?
- Define brand awareness, sales, education, entertainment, portfolio)
- Set SMART goals (Specific, Measurable, Achievable, Relevant, Time bound)

2. Identifying Target Audience

- Who will use the site? (Students, customers, professionals etc.)
- Understand their needs, age group, technical skills and expectations

3. Research & Competitor Analysis

- Analyze similar websites to learn best practices
- Identify what works well and what can be improved

4. Decide on Website Type & Features

- Static (Informational) or Dynamic (Interactive)?
- What features are needed? (Login, forms, blog, gallery etc)

- Decide who will update content and maintain the site after launch

5. Content Planning

- Plan what content will go on each page
- Decide tone of writing (formal, friendly, technical)
- Collect text, images, videos, graphics

6. Site Structure & Sitemap (Sketch)

- Organize pages in hierarchy (main menu, subpage)
- Create a sitemap showing how pages are connected

7. Layout Design

- Sketch basic page layouts before coding
- Place headers, footers, menus, sidebars and content area
- Decide color scheme, fonts & branding style

8. Technology Selection

- Choose front-end (HTML, CSS, JavaScript, React, etc)
- Choose back-end (PHP, Python, Node.js, etc)
- Select database (MySQL, MongoDB etc)
- Choose CMS (WordPress, Joomla, custom-built).

9. Budgeting & Timeline

- Estimate cost of development, hosting, domain, maintenance
- Set deadlines for each phase (design, coding, testing, launch)

10. Testing & Maintenance Plan

- Plan for testing (usability, performance, security)

Five Golden Rules of Web Designing

- Keep design simple and consistent
- Why it matters: Simplicity enhances usability and improves the user experience
- How to apply: Use a clean layout, limit fonts and colors and maintain consistency in navigation, headings, and spacing across pages

③ Focus on Loading Speed

- Why it matters: Slow websites drive users away and hurt SEO (search engine optimization) ranking
- How to apply: Optimize images, minify CSS/JS files and use fast, reliable hosting

③ Ensure mobile and cross-browser compatibility

- Why it matters: Users access sites from various devices (mobile, tablet, desktop etc) and browsers
- How to apply: Use responsive design that works on different platforms

④ Focus on User Experience (UX)

- why it matters: A positive UX keeps visitors engaged and encourages return visits
- how to apply: Understand user needs, create clear paths and minimize friction in interactions

⑤ Make Navigation Easy and Intuitive

- why it matters: Users should be able to find what they need quickly and without confusion
- how to apply: Use clear menus, breadcrumbs, and logical site structure. Place "important links" in expected locations

Designing a Navigation Bar

1. - Use clear and concise labels:
 • keep link names short & easy to read (e.g., "Home", "About", "Services").
 • Avoid long phrases

Ex:- `<a href = "#"> Home </a>`

`<a href = "#"> Contact </a>`

2. Keep it consistent across all pages

- same order / same style on every page
- don't shuffle link positions between pages

3. Highlight current page

- show users which page they are on

Ex:-

• active {
 background-color: #555555;

4. Use drop-downs for subcategories (if needed)

Under "Services", add "Web Design", "SEO etc"

`<div class = "dropdown">`

`<a href = "#"> Services </a>`

`<div class = "dropdown-content">`

`<a href = "#"> Web Design </a>`

`<a href = "#"> SEO </a>`

`</div>`

5. Ensure accessibility

- Use proper contrast (white text on dark background)
- add alt, aria-label for screen reader & using icons
- make links large enough to click easily

more focus on color

Page Design / Design concept

1. Main/visual hierarchy

- title → Sub heading → Paragraph → Images
- Bg headings grab attention, smaller text for details

2. Use whitespace effectively

- Don't overcrowd text/images
- Leave breathing space around sections

3. Consistent fonts and color scheme

- same font family throughout
- use brand colors (e.g. dark heading, light background)

4. Prioritize content readability
 - Short paragraphs, simple words,
 - Font size at least 16pt for body text

5. Ensure responsive layout

- Should look good on mobile & desktop.
- Use relative units (% , em, rem) instead of fixed px.

Example ->

Home Page Layout

Website layout

1. Highlight main services / products

Show what you offer right away (Cards, Icons, Show titles)

2. Include a call to action (CTA)

Ex:- "Contact Us", "Get started", "Buy Now"
Buttons should stand out in color.

3. Use eye-catching banners or sliders

A big hero image at the top with a strong message

4. Brief introduction / about section

Small paragraph introducing your brand

5. Link to major sections of the site

Quick navigation to services, about, contact etc.

Layout design - Arrangement of elements like

header

footer

body

sidebar

II -> Basics in Web Design

Design Concept

IA-2

1. Theme Selection

Decide the overall "feel"; professional, playful, minimal, creative etc

Ex:- Tech startup -> modern + clean

Barber -> warm + friendly

2. Color palette and typography

- Pick 2-3 main colors (primary, secondary, accent)
- Choose 1-2 fonts (one for headings, one for body text)

Ex:- Blue + white + orange with a bold heading font

3. Brand identity integration

- Use your logo, brand colors and tone of voice
- Design should "look like your brand", not generic

4. Consistency in UI/UX (user experience)

- Buttons, headings, icons should look the same across all pages
- Ex:- same button style for "Sign Up" and "Learn More"

"Learn More"

5. Design for accessibility

- Good contrast (dark text on light background)
- Use alt text for images
- Make font size readable (16pt+)

Brief History of Internet

1. Originated from ARPANET (1969)

- The internet began as ARPANET, a U.S. government project that connected universities and research labs to share data over long distances.
- It successfully demonstrated that computers could communicate over a network.

(email started in 1971 by ~~Robert~~ Ray Tomlinson (communication system))

2. TCP/IP protocol introduced in 1983, this started in 1984

- TCP/IP became the standard communication protocol for all connected devices.
- On January 1, 1983, ARPANET officially switched to TCP/IP, allowing different types of computers to communicate across a unified network.

network

3. WWW invented in 1989 by Tim Berners-Lee

(1989-1990)

Tim Berners-Lee developed the world wide web as a way to access and share information through linked documents using browsers.

- He introduced the components like HTML, HTTP and URLs.

4. Rapid expansion in the 1990s

- The internet saw massive growth during this decade as it became available to the public.
- Web browsers, search engines and e-commerce websites emerged, making internet a global phenomenon.

1. Widespread use with broadband and mobile tech

- In the 2000s and beyond, faster broadband connections and the rise of smartphones made the internet available almost everywhere.
- The led to daily use of the web for communication, entertainment and work.

What is World Wide Web?

- World Wide Web (WWW) is a system of interlinked hypertext documents and resources that can be accessed via the internet using a web browser.
- The world wide web is a collection of information and services, connected by links that runs on the internet.
- To access and share information through websites, webpages, images, videos and other multimedia content.

Why Create a Website

1. Increase Visibility and Reach
 - A website makes business accessible worldwide.
 - Anyone searching online can find products, services or information anytime.
2. Build Credibility and Trust
 - It helps gain customer confidence through professional reviews, testimonials and a polished online presence.
3. Showcase Products/Services
 - Website acts as digital portfolio or catalog.
 - It allows potential customers to see products, services pricing or work samples easily.

4. Enable Communication and Interaction

- Websites provide ways for visitors to contact
- Ask questions of the product or engage with content

5. Monetization and Marketing (To make money)

- A website can help to earn money directly (selling products / services online) or indirectly (advertising, affiliate marketing or promoting brand)
- It is also a cost-effective marketing tool that reaches a large audience.

Web Standards

→ web standards are rules created by W3C to ensure websites are properly displayed

1. Set by W3C

- W3C is the main organization that creates and maintains web standards.

• They make sure website follows the same rules so they work correctly across all browsers and devices worldwide.

2. Includes HTML, CSS, JavaScript standards

- web standards cover the key technology of the web
- ~~HTML~~ - standards ~~with~~ the key technology
- HTML - structure / content on a page
- CSS - styling the layout, colors and fonts
- JavaScript - adds interactivity & dynamic features

3. Ensure compatibility and accessibility

- web standards make websites work well on different browsers, devices and screen sizes
- They also make sites usable for all people to ensuring a wider audience can access the content

Audience Requirements

1. Identify the Audience

- Determine who will see your site: customers, students or general users.
- Consider their age, gender, location, education, interests and technical skills

2. Understand Their Goals

- Figure out what visitors want to achieve: buying products, learning information, contacting or getting service
- Align website features and content with these goals.

3. Device and Format

- Know how audience will access the website: mobile, tablet or desktop, primary device
- Design for easy navigation, fast loading and a smooth experience on all devices

4. Focus on Purpose and Usability

- Choose design style, content type, and layouts that appeal to audience
- Ensure the site is intuitive and easy to use.

5. Accessibility

- Make the website usable for everyone, including people with disabilities
- Include screen reader compatibility, proper color contrast and readable fonts

XML and Web Hosting

What is XML?

XML - (Extensible Markup Language) is a markup language designed to store and transport data.

- It is both human readable and machine-readable.
- Unlike HTML, which focus on displaying data, XML focus on describing and organizing data.

XML is a flexible way to store, organize and share data across platforms and applications.

(Use the language Java (Browser support))

Key Features of XML

- ① Extensible → you can create your own custom tags
- ② Structured Data → store data in a tree-like structure with opening and closing tags
- ③ Self descriptive → The tags explain what the data means
- ④ Platform independent → works across different systems and applications
- ⑤ Separation of data and presentation → XML only holds data, it doesn't decide how it looks.

Example:

```
<Student>
  <Name> Rahul </Name>
  <Age> 21 </Age>
  <Course> BCA </Course>
</Student>
```

in XML
all tags are similar
in XML
all tags are similar
in XML
all tags are similar

Key difference between HTML and XML

HTML	XML
① HTML's primary purpose is to display and present data to a user in a web browser	① XML's primary purpose is to store and transport data
② HTML uses a predefined set of tags like <p> 	② XML is an extensible markup language to create tags to describe the data.
③ HTML is generally not case-sensitive and is very forgiving of errors	③ XML is case-sensitive as a single error will cause the document to fail
④ HTML documents have a flexible structure	④ XML documents are well-formed and must be validated against a schema

Defining XML tags

- Tags in XML are used to mark or describe data
- Each piece of data is wrapped inside a pair of opening and closing tags
- XML documents have predefined tags like HTML
- We can create our own tags based on the data want to store

Document Type Definition (DTD)

- Rules for defining tags and closing tags
- Tags must have an opening / tag and closing / tag
- Tag names are case-sensitive (<Name> ≠ <name>)
- Tag must start with numbers or special characters
- Tags must be properly nested

Defining attributes and values in XML tags

- What are attributes?
- Attributes provide info information about an element
- Attributes appear as name-value pairs
- Attributes must be unique within a tag
- Values must always be enclosed in quotes (" ")
- Attributes cannot ignore child elements - they are just extra details
- Attributes names are case sensitive.

```
<Student id="101" branch="BCA">
  Name? Rohan /Name?
  Age? 20 /Age?
  </Student>
```

- In XML, a Document Type Definition (DTD) defines the structure and the legal elements and attributes of an XML document.
- It acts as a set of rules to validate the XML file contents

Two types

- Internal DTD - defined within the XML document
- External DTD - defined from an external file.

1 Internal DTD example

```
<?xml version="1.0"?>
<!DOCTYPE note [
  <!ELEMENT note (to, from, heading, body)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
```

<note>

```
<to> Rohan </to>
<from> Jansi </from>
<heading> Reminder </heading>
<body> Don't forget me this weekend! </body>
```

- Here the DTD is inside the XML file (between <!DOCTYPE and <...>).
- It defines a root element <note> that must contain the elements to, from, heading and body
- Each of these child elements contains paired

② External DTD Example.

gave this file as separate file like note.dtd

```
<!ELEMENT note (to, from, heading, body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

Save this file as separate file like first.xml

```
<?xml version="1.0"?>
<doctype note system="note.dtd">
<note>
  <to> Rohit </to>
  <from> Jassi </from>
  <heading> Reminder </heading>
  <body> Don't forget me this weekend! </body>
</note>
```

XML Schema (XSD) <?xml>

An XML Schema (usually written in a file ending with .xsd) defines the structure, data types, and rules of an XML document - similar to a DTD

• It uses XML syntax itself

• use • define which elements and attributes appear

• specify the datatype (string, integer, date, etc.)

• sets restrictions (like min/max values, required elements)

Ensuring data is valid and consistency

blue schema and dtd is datatype is in schema

<html>

<head>

<link rel="stylesheet" href="a.css">

</head>

<body>

<nav bar class="nav">

 home

 about

 service

</body>

</html>

automatic move image - section

here -> page display

how -> user move as mouse clicking

of -> with word

example ->

of -> with word

example ->

of -> with word

example ->

of -> with word

example ->

of -> with word

example ->

of -> with word

example ->

of -> with word

example ->

example for XML Schema (XSD)

example: student.xml

<?xml version="1.0"?>

<student xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="student.xsd">

<name>Rahul</name>
<age>20</age>
<marks>88</marks>

</student>

student.xml

<student>

<name>age</name>

<marks>

xsi:noNamespaceSchemaLocation="student.xsd"

links the XML to its schema file.

example: student.xsd

<?xml version="1.0"?>

<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema" >

<xsd:element name="student">

<xsd:complexType>

<xsd:sequence>

<xsd:element name="name" type="xsd:string">

<xsd:element name="age" type="xsd:integer">

<xsd:element name="marks" type="xsd:integer">

</xsd:sequence>

</xsd:element>

</xsd:schema>

<xsd:element name="student">

<xsd:complexType>

<xsd:sequence>

type="xsd:string"/>

- XHTML Parsing → Reading & analyzing XML
- HTML or Extensible HyperText Markup Language is a mix of HTML and XML, very similar to HTML but stricter
- It's like a rulebook for creating web pages that browsers easily understand.

Rules

- All tags must be properly closed (
 instead of
).
- Tags are case sensitive (<body> not <BODY>).
- All attributes must be quoted (src="image.jpg").
- The document must be well-formed

Why XML Parsing is needed (used)

- 1) XML documents are validated with standard XML tools
- 2) It is easy to maintain, convert, edit document in the long run
- 3) It is used to define the quality standard of web page
- 4) XML is an official standard of the W3C website
- 5) XML is more compatible and accurate with many browsers.

Example
XML version = "1.0" encoding = "UTF-8" ?

<?xml xmlns = "https://www.w3.org/1999/xhtml" ?>

<html> Sample Page </html>

</html>

</body>

<h1> Welcome </h1>

<p> This is a paragraph </p>

</p>

</body>

</html>

Methods of Parsing XML [Types of XML]

Method	Description	Explanation
1) DOM / Document Object Model	creates a tree structure of entire document in memory	easy to access and modify elements
2) SAX (Simple API for XML)	Reads XML file line by line and triggers events	less memory usage, good for large files

Comparison of XML

DOM is better than SAX for advantages like, better structure, better Java example

Feature	DOM Parser	SAX Parser
1) Memory Usage	Loads the XML -> high memory	Reads sequentially -> low memory
2) Random access	Yes	No
3) Modification	can modify XML	cannot modify XML
4) Speed	Slower for large XML	Faster for large XML
5) Complexity	easy	Slightly complex
6) But For	small XML, modify -> random needed	large XML, read-only

Document Object Model (DOM)

DOM (Document Object Model) is a programmatic representation of an XML (or HTML) document.

It treats every part of the XML - elements, attribute, text - as a node in a tree structure

Using DOM, we can read, modify, add or delete elements in XML with programming language like Python, Java or JavaScript

Advantages of DOM

- 1) easy navigation - Move through parents, child or sibling nodes
- 2) modify XML dynamically - we can change text, attribute or add/delete nodes
- 3) language-independent - works with Python, Java, JavaScript, etc.

Example:-

<bookstore>

<book category = "reading">

<title> Everyday Indian </title>

<author> Nachik Das </author>

<price> 30.00 </price>

</book>

</bookstore>

structure.

bookstore (root)

book (attribute: category = "reading")

title (txt: "Everyday Indian")

author (txt: "Nachik Das")

price (txt: "30.00")

Example for parsing

DOM Parser Example: students.xml

<students>

<student roluno = "101"><name> Ramesh </name>

<student roluno = "102"><name> Sita </name>

</students>

Example for java code DOM

import javax.xml.parsers.*;

import org.w3c.dom.*;

import java.io.File;

public class MinimalDOM

{ public static void main(String[] args) throws Exception

{

Document doc = DocumentBuilderFactory.newInstance().

newDocumentBuilder().

parse(new File("students.xml"));

NodeList students = doc.getElementsByTagName("student");

for (int i = 0; i < students.getLength(); i++) {

Element s = (Element) students.item(i);

System.out.println(s.getAttribute("roluno") + " " +

s.getAttribute("name") + " " +

s.getAttribute("price"));

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

Example: students.xml

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

```
<?xml version="1.0" ?>
<students>
  <student name="John">
    <name>John</name>
    <id>1</id>
  </student>
  <student name="Jane">
    <name>Jane</name>
    <id>2</id>
  </student>
</students>
```

Introduction to Web Publishing or Hosting:

(Moving the web site going online with a server)

• Web hosting: Web hosting is a service that provides space and resources needed to store website files (like HTML, images, code etc) on a specialized computer called a web server.

• Storing website files on a server so anyone can access them online.

• To make website files available to the world wide web (www).

Types of web hosting:

① Shared hosting: Multiple website share a single server resource. Best for small, low-traffic sites.

② VPS (Virtual Private Server) Hosting: A single server is partitioned into several private virtual servers offers more control and resources than shared hosting.

③ Dedicated hosting: Renting an entire physical server for a single website. Best for large, high-traffic sites.

④ Cloud hosting: Use a network of virtual servers, allowing for great scalability and flexibility.

Creating the Website.

① Step 1: Create website File

- Determine website is for (blog, business, e-commerce, portfolio, etc) and create the visual design
- Write the website files using languages like HTML, CSS, and JavaScript etc.
- Use a drag-and-drop platform like Wix, Squarespace or Google Site
- Use software like WordPress, which allows to manage content easily without extensive coding

② Step 2: Test Locally

- Open index.html in browser to check the website.
- Make sure all links, images and styles are working.

③ Step 3: Choose Hosting

- Register a Domain Name: Choose and purchase unique web address from a domain registrar.
- Select a hosting provider like GitHub Pages, Netlify, Firebase Hosting for Free Hosting or GoDaddy, HostGator, Bluehost for Paid Hosting

④ Step 4: - Upload File

- Create a domain provider account and login.
- For hand-coded site, you typically use an FTP (File Transfer Protocol) client to transfer your local files to the remote web server.

⑤ Step 5: - Connect Domain to Hosting (DNS configuration) and Launch

- Updating domain name in the Name Servers in domain registrar's settings. This tells the Internet where your website files are located.

check all links, forms and functionality across different browsers and devices (especially mobile).
 Once everything is verified, your website is officially published and live!

Saving the Website

"Saving the website" can refer to two main actions:
 1. Saving the source files to local computer during development.

① Saving from a Live Host

1. Saving from your local development environment.
 When we are developing or managing a website, we need to keep a complete organized copy of all its files on computer.

If we are creating the website on computer, the "saving" process is simply using text editor or IDE (Integrated Development Environment) to save our work.

Organize:- Create a single root folder for entire website.

Structure:- Inside the root folder, keep main files and clearly named sub-folders:

- index.html (your homepage file)
- css/ (for stylesheets)
- js/ (for JavaScript files)
- images/ (for all media and pictures)

Save:- Use the save command (Ctrl+S) in editor to save changes to manually.

Website Management

② Saving from a Live Host (Backup/Download)

To save a backup of a live website hosted on server, typically use FTP client or host's control panel.

Use an FTP client (Example: FileZilla):

— Connect to web host using the FTP credentials (Host, Username, Password).

— Navigate to website's root directory on the server (often named public_html or www).

— Download the entire directory structure to a folder on local computer. This gives all the HTML, CSS, JavaScript and image files.

Download the Database (for Dynamic Sites):

Many websites use a CMS like WordPress (which relies on a database) we must also export the database file (usually a .sql file) using host's control panel tool, like phpMyAdmin.

Working on the Website

① Security & Stability

Regularly backup all website files and the database. Install security patches and software/plugin updates (especially for CMS sites like WordPress) Monitor for malware or suspicious activity.

② Content & SEO

Publish high content (blog posts, news, update old content to keep it relevant and relevant. Dedicate fix broken links (404 errors). Optimize pages for better search engine rankings (SEO).

③ Performance & Testing

Monitor site speed and loading time. Optimizing images is a common task; regularly test all key pages, especially contact forms and e-commerce checkout process.

④ Analysis & Improvement

Use tools like Google Analytics to monitor visitor traffic & behavior. Analyze which pages are working well and which ones need design or content improvement.

Website Structure - Arrangement of pages and linking

- Creating a website structure, often called Information Architecture (IA) is a foundational step in website development.
- It involves organizing, labeling and defining the relationships between all the pages and content on site.

• The most common and user-friendly structure is the Hierarchical (or Tree) Structure

- Homepage (the root) is at the top
- Main categories (services, about us, Blog)
- branch out from the homepage.
- subpages (specific services, individual blog posts) branch out from their respective categories.

Creating Titles for webPage.

Key characteristics of a creating page title

- ① Unique:- Each page should have a distinct title
- ② Concise:- Aim for 50-60 characters so the title is fully visible in search results
- ③ Descriptive:- The title should clearly state what the page is about.
- ④ Relevant:- The title must accurately reflect the content of the page.
- ⑤ Incorporate keywords:- Include your main keyword naturally, as titles are crucial factor for SEO.

Website Theme - overall design style - e.g. colors, fonts, layout

Website Publishing - making website live on the internet

Theme Publishing Websites -> m

- ① Responsive Design:- The layout must automatically adjust and look great on all devices (desktop, tablet, mobile) for optimal user experience and SEO.

- ② Clear Navigation:- Intuitive menus, clear categories and effective search functionality to help visitors find content quickly

- ③ Content Organization:- Supports diverse content blocks, multiple column layouts, featured post slides and category labels to present a high volume of articles attractively

- ④ Readability:- Excellent typography, clear font, sufficient line height and white space to ensure long-form articles are easy to read.

- ⑤ Monetization options:- Built-in dedicated space for advertisements (banners, native ads) that are easy to manage without breaking design.
- ⑥ Speed / Performance:- Optimized to load quickly, as publishing sites often have many images, videos and complex layouts
- ⑦ Social Integration - Easy-to-use social sharing buttons and options to embed social media content to increase reach.
- ⑧ SEO optimization:- Clean well-coded structure that helps search engines crawl and index content effectively
- ⑨ Author Bios / Archive - Feature to prominently display author information and dedicated page for all their articles.